

08 软件设计与体系结构 — 复习纲要

一、考试概况

题型	题量
单项选择题	20 道
判断题	10 道
填空题	10 道
简答题	6 道
应用题	3 道

考试范围：第 1~7 章

二、应用题重点（第 3、4、7 章）

第 3 章 经典软件体系结构风格

3.2 数据流风格（p59 - 62）

（1）批处理风格

基本思想：将数据处理的各个步骤作为独立的处理组件，每个步骤完成一项特定任务，前一步骤的输出作为后一步骤的输入

组件：独立的数据处理程序

连接件：数据文件或共享存储介质

特点：各处理步骤按预定顺序依次执行，数据以整体批量的方式传递

（2）管道 - 过滤器风格

基本思想：将系统功能分解为一系列过滤器组件，通过管道连接件将数据流从一个过滤器传递到下一个过滤器

组件：过滤器（Filter）— 对输入数据进行处理并产生输出

连接件：管道（Pipe）— 在过滤器之间传输数据流

特点：

过滤器是独立的实体，不共享状态

各过滤器可以增量地处理数据

支持并发执行

可复用、可重组

（3）两者对比

	批处理风格	管道 - 过滤器风格
数据传输方式	整体批量传递	增量流式传输
组件耦合度	低	低
并发性	不支持	支持
适用场景	延迟要求低的离线处理	需要渐进式输出的在线处理

3.3 基于事件的内式调用风格（p66 - 67）

基本思想：组件不直接调用其他组件的方法，而是发布（广播）事件；其他组件通过注册事件处理过程来响应事件

核心机制：

事件源：产生事件的组件

事件监听器：注册了对某一事件的响应过程

事件分发器：负责将事件传递给所有已注册的监听器

优点：组件解耦，可扩展性强，支持灵活的系统演化

缺点：事件传递的先后顺序不明确，调试困难，性能开销

第 4 章 分布式软件体系结构风格

4.2 两层 C/S 体系结构风格

客户端（Client）：负责用户界面展示和部分业务逻辑

服务器端（Server）：负责数据存储和管理，通常为数据库服务器

连接件：网络通信协议（如 TCP/IP）

优点：数据集中管理，客户端响应快

缺点：客户端维护和升级困难（胖客户端问题），扩展性受限

4.3 P2P 体系结构风格

基本思想：网络中每个节点地位对等，既是服务的提供者也是消费者

特点：

去中心化，无集中式服务器

各节点自治，可动态加入或离开

系统具有高度容错性和可扩展性

典型应用：文件共享（BitTorrent）、区块链、即时通信

4.4 三层 C/S 体系结构风格

表示层：用户界面，负责与用户交互

业务逻辑层：核心业务处理，运行在应用服务器上

数据层：数据存储和访问，运行在数据库服务器上

优点：

各层独立开发和维护

业务逻辑集中管理，便于复用

比两层 C/S 更易扩展

与两层 C/S 的关键区别：将业务逻辑从客户端分离出来成为独立一层

4.5 B/S 体系结构风格（三层）

浏览器（Browser）：表示层，通过 Web 浏览器访问系统

Web 服务器/应用服务器：业务逻辑层

数据库服务器：数据层

优点：

客户端零部署、零维护

跨平台，只需浏览器即可访问

便于系统升级和扩展

与三层 C/S 的关系：B/S 是三层 C/S 在 Web 环境下的特例

第 7 章 设计原则

7.1 开闭原则（OCP, Open-Closed Principle）

定义：软件实体（类、模块、函数等）应当对扩展开放，对修改关闭

含义：当需求变化时，应通过扩展已有代码（如新增子类）来实现新功能，而非修改已有代码

实现方式：使用抽象和多态，定义稳定的抽象接口，具体实现可变

实例要点：通过抽象基类定义接口，新增功能时只需添加新的子类实现，无需修改已有类的代码

7.3 依赖倒转原则（DIP, Dependency Inversion Principle）

定义：

高层模块不应依赖低层模块，二者都应依赖抽象

抽象不应依赖细节，细节应依赖抽象

含义：面向接口编程，而非面向实现编程

实现方式：通过接口或抽象类来声明依赖关系，具体实现类通过依赖注入等方式提供

实例要点：高层模块依赖于接口定义，低层模块实现该接口，依赖关系从“高层→低层”倒转为“高层→抽象←低层”

三、简答题重点（第 2、3、6 章）

第 2 章 面向对象方法

2.2 软件体系结构 — 组件与连接件的定义

组件（Component）：软件体系结构中具有独立功能的计算单元或数据存储单元，是系统功能的承载者

组件定义了自身的接口（提供和请求的服务）

组件封装了内部实现细节，对外仅通过接口交互

连接件（Connector）：在组件之间建立交互关系、传递消息或数据的机制

连接件定义了组件间交互的规则和约束

常见的连接件类型：过程调用、消息传递、管道、事件广播、共享数据访问等

第 3 章 经典软件体系结构风格

3.2 数据流风格（批处理风格、管道 - 过滤器风格）

批处理风格：

数据以整体文件形式在组件间依次传递

每个组件处理完整个数据集后才传递给下一个组件

适用于数据量大的离线计算场景

管道 - 过滤器风格：

数据以流的形式增量传递

各过滤器可以并发执行

适用于需要实时或渐进式输出的场景

典型实例：Unix/Linux 中的管道命令（如 `cat file | grep keyword | sort``）

3.6 解释器风格

基本思想：为某种语言（或表达式）构建一个解释引擎，逐条读取、分析并执行

核心组件：

解释引擎：负责解析和执行

程序状态：存储当前执行上下文和变量值

伪代码/规则库：被解释执行的程序

适用场景：规则引擎、脚本解释器、虚拟机

优点：灵活性高，易于扩展语言功能

缺点：执行效率低于编译执行

3.7 反馈控制环风格

基本思想：系统通过持续监测被控对象的输出，与期望值进行比较，根据偏差调整控制输入，使系统输出趋近期望值

核心组件：

传感器（Sensor）：采集被控对象的当前输出值

控制器（Controller）：计算偏差并产生控制信号

执行器（Actuator）：根据控制信号调节被控对象

被控对象：被控制的物理系统或软件系统

控制原理：负反馈 — 将偏差反向作用于系统输入以减小偏差

适用场景：自适应系统、自动驾驶、恒温控制、负载均衡

第 6 章 软件设计目标

好的软件设计应具备以下目标：

1. 正确性（Correctness）：软件系统满足规格说明和用户需求，执行结果符合预期

2. 健壮性 (Robustness)：系统在异常输入或异常运行条件下仍能正常运行或优雅降级
3. 高效性 (Efficiency)：合理利用时间和空间资源，满足性能要求
4. 可复用性 (Reusability)：软件组件可在不同系统或不同场景中重复使用
5. 可维护性 (Maintainability)：软件易于理解、修改、测试和扩展
6. 可扩展性 (Scalability)：系统能够适应需求变化和规模增长
7. 可靠性 (Reliability)：系统在规定条件和时间内无故障运行的能力
8. 可移植性 (Portability)：软件可在不同平台或环境间迁移

四、其他考点

第 1 章 概述 — 结构化方法与面向对象方法

结构化方法

概述：以功能分解为核心，自顶向下、逐步求精地将系统分解为若干功能模块

要素：数据流图 (DFD)、数据字典 (DD)、结构图 (SC)、处理逻辑描述

优点：

方法成熟，文档规范

适合需求明确、规模适中的项目

缺点：

重用性差，维护困难

难以应对需求变化

数据与操作分离，不符合人类认知习惯

面向对象方法

概述：以对象为核心，将数据与操作封装在一起，通过对象间的消息传递实现系统功能

要素：对象、类、封装、继承、多态、消息

优点：

符合人类认知习惯

可复用性好

可维护性好

能有效应对需求变化

缺点：

学习曲线较陡

某些场景下运行效率略低

第 2 章 — 用例图、类图、软件开发模型

用例图

描述外部执行者（Actor）所理解的系统功能

用于需求分析阶段，捕获系统的功能需求

元素：参与者（Actor）、用例（Use Case）、关系（包含 <<include>>、扩展 <<extend>>、泛化）

类图

描述系统中类的静态结构及其相互关系

是面向对象建模的核心图

关系类型：

关联（Association）：类之间的语义连接

聚合（Aggregation）：整体 - 部分关系（弱拥有，部分可独立存在）

合成/组合（Composition）：整体 - 部分关系（强拥有，部分不能独立存在）

依赖（Dependency）：一个类使用另一个类

泛化（Generalization）：继承关系

常见软件开发模型

模型	特点
瀑布模型	各阶段线性推进，阶段间有反馈
增量模型	分批交付功能递增的产品
螺旋模型	结合原型与风险分析
迭代模型	反复细化，每次迭代产生可运行版本
渐进式开发模型	逐步完善，持续交付
变换模型	基于形式化规格的自动/半自动转换

第 3 章 — 软件体系结构风格补充

3.1 主程序 - 子程序风格

基本思想：系统由一个主程序和若干子程序组成，主程序负责控制流程，子程序执行具体功能

特点：

自上而下的层次化分解

以功能执行为驱动

适合批处理和命令行程序

缺点：

不适合图形用户界面程序开发

可重用性差

数据安全性差（全局数据共享）

3.1 面向对象风格的特点

数据及操作封装在对象内部

对象间通过消息传递进行交互

支持继承和多态

便于建模现实世界

第 5 章 — MVC 风格与 Struts 风格

5.1 MVC 风格概述

M (Model, 模型)：业务数据和业务逻辑

V (View, 视图)：用户界面展示

C (Controller, 控制器)：接收用户输入，调用模型，选择视图

交互流程：

用户操作 → Controller → 更新 Model → Model 通知 View → View 刷新显示

优点：

模型与视图分离，一个模型可对应多个视图

提高代码复用性和可维护性

便于团队分工 (UI 与业务逻辑分离)

Struts 风格

基于 MVC 模式的 Java Web 框架

使用 ActionServlet 作为前端控制器

通过配置文件实现页面流转

第 6 章 — 软件设计目标（补充要点）

健壮性：系统遇到异常输入或环境异常时不崩溃，能给出合理提示或降级运行

高效性：在满足需求前提下，尽可能减少时间和空间开销

选择合适的算法与数据结构

避免不必要的复制

可复用性：设计时应考虑组件的通用性，使同一组件可在多个场景使用

遵循设计原则（如开闭原则、接口隔离）

可维护性：代码清晰、结构合理、注释充分

便于定位和修复缺陷

便于添加新功能

第 7 章 — 设计原则（补充）

7.1 开闭原则（OCP）

对扩展开放，对修改关闭

实现手段：抽象 + 多态

7.2 里氏代换原则（LSP）

定义：子类型必须能够替换其基类型

含义：任何使用基类引用的地方，都可以透明地使用子类对象，且程序行为不变

要求：子类不应改变父类方法的预期行为（不能违背父类方法的前置条件和后置条件）

实例要点：正方形继承长方形时，若修改 `setWidth/setHeight` 的行为导致违反父类约定，则违背 LSP

7.3 依赖倒转原则（DIP）

高层模块与低层模块都依赖抽象

抽象不依赖细节，细节依赖抽象

五、重点章节速查(可能)

章节	主要内容	重点题型
第 1 章	结构化方法、面向对象方法概述与对比	选择题、判断题
第 2 章	用例图、类图、软件开发模型、组件与连接件定义	选择题、简答题
第 3 章	数据流风格、隐式调用、解释器、反馈控制环、主程序-子程序	应用题、简答题
第 4 章	两层 C/S、P2P、三层 C/S、B/S	应用题
第 5 章	MVC 风格、Struts 风格	选择题、判断题
第 6 章	设计目标（正确性、健壮性、高效性等）	简答题
第 7 章	开闭原则、里氏代换、依赖倒转	应用题